Practical No 1

Aim: - Write a Program to demonstrate the following aspects of signal

processing on suitable data:

1. Upsampling and Downsampling on Image/Speech Signal.

2. Fast Fourier Transform to compute DFT.

Code :-

```
from PIL import Image
from skimage.io import imread, imshow
import numpy as np
import matplotlib.pyplot as pylab
import numpy.fft as fp
from skimage.color import rgb2gray

im = Image.open(r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-
photo.jpg")

im1 = im.resize((im.width * 5, im.height * 5), Image.Resampling.NEAREST)

im2 = im.resize(
   (max(1, im.width // 15), max(1, im.height // 15)),
   Image.Resampling.LANCZOS
)

freq = fp.fft2(rgb2gray(np.array(im.convert("RGB"))))

pylab.figure(figsize=(10, 14))

pylab.subplot(2, 2, 1)
pylab.imshow(im)
pylab.title('Original Image', size=20)

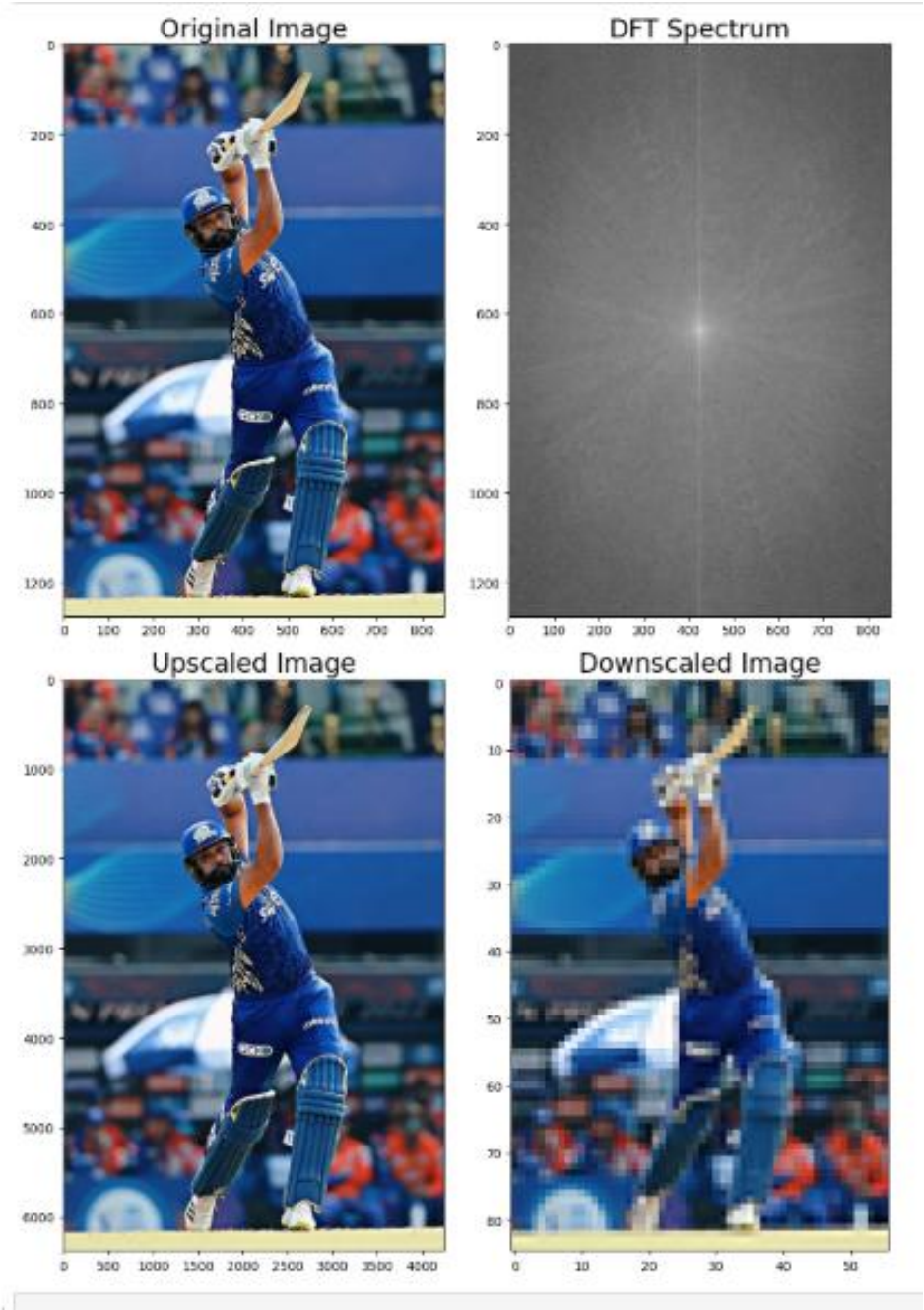
pylab.subplot(2, 2, 2)
pylab.imshow(20 * np.log10(0.01 + np.abs(fp.fftshift(freq))), cmap='gray')
pylab.title('DFT Spectrum', size=20)

pylab.subplot(2, 2, 3)
pylab.imshow(im1)
pylab.title('Upscaled Image', size=20)

pylab.subplot(2, 2, 4)
pylab.imshow(im2)
pylab.title('Downscaled Image', size=20)
```

```
pylab.tight_layout()
pylab.show()
```

Output



Practical No 2

Aim: - Write program to perform the following on signal:

1. Create a triangle signal and plot a 3-period segment.

2. For a given signal, plot the segment and compute the correlation

between them.

Code :

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

frequency = 1
duration = 3
sampling_rate = 1000

num_samples = int(duration * sampling_rate)
t = np.linspace(0, duration, num_samples, endpoint=False)

triangle_signal = signal.sawtooth(2 * np.pi * frequency * t, width=0.5)

plt.figure(figsize=(8, 4))
plt.plot(t, triangle_signal, label='Triangle Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('3-Period Segment of Triangle Signal')
plt.xlim(0, 3 / frequency)
plt.grid(True)
plt.show()

square_signal = np.sign(np.sin(2 * np.pi * frequency * t))

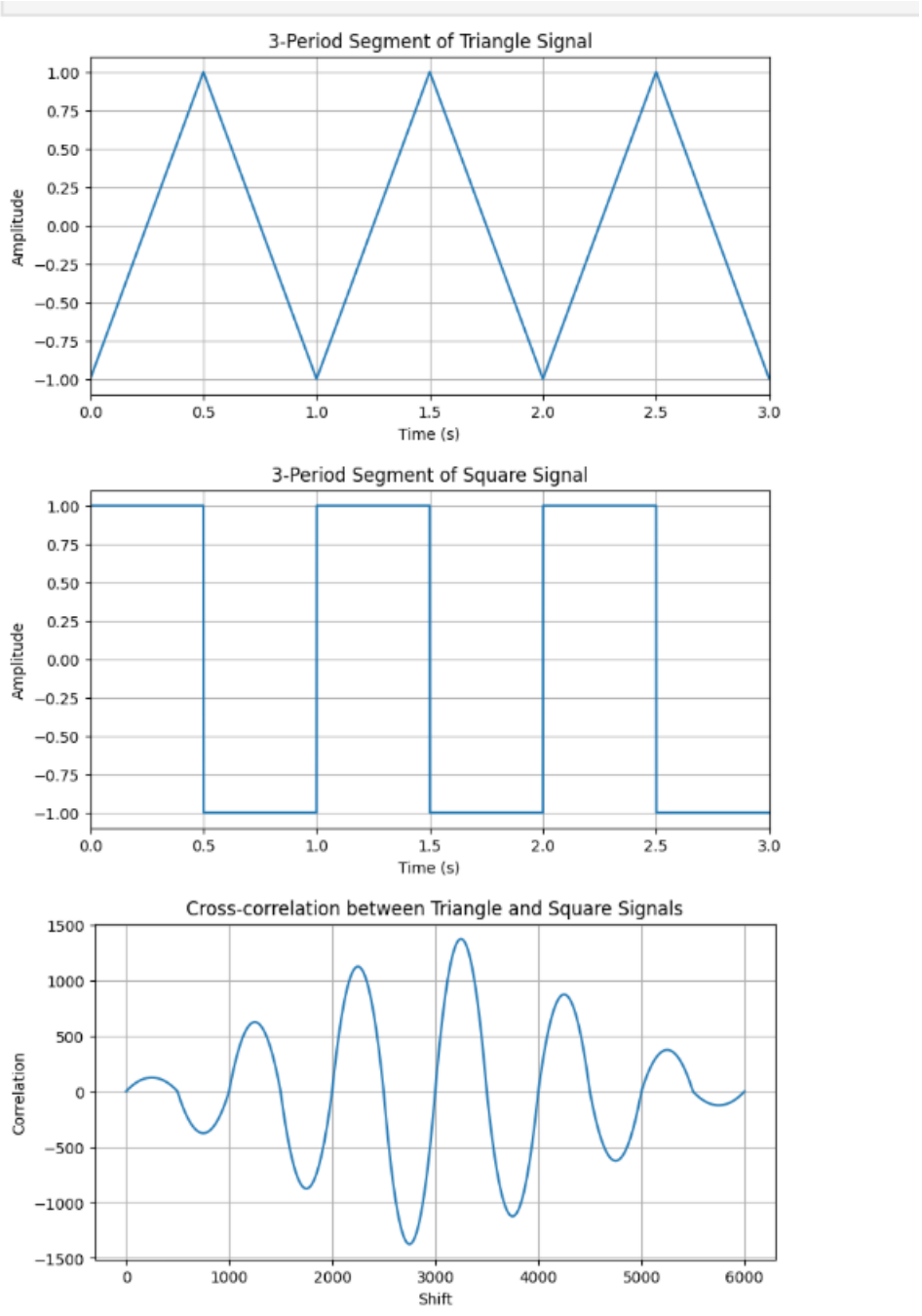
plt.figure(figsize=(8, 4))
plt.plot(t, square_signal, label='Square Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('3-Period Segment of Square Signal')
plt.xlim(0, 3 / frequency)
plt.grid(True)
plt.show()

correlation = np.correlate(triangle_signal, square_signal, mode='full')

plt.figure(figsize=(8, 4))
plt.plot(correlation)
plt.xlabel('Shift')
plt.ylabel('Correlation')
```

```
plt.title('Cross-correlation between Triangle and Square Signals')
plt.grid(True)
plt.show()
```

Output :



Practical No 3

Aim: - Write program to demonstrate the following aspects of signal on

sound/image data

1. Convolution operation

2. Template Matching

Code :

```
import numpy as np
import matplotlib.pyplot as plt
import skimage as ski
from scipy.signal import convolve2d

image = ski.color.rgb2gray(
   ski.io.imread(r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-
photo.jpg")
)

template = image[390:445, 430:530]

result = ski.feature.match_template(image, template)
ij = np.unravel_index(np.argmax(result), result.shape)
x, y = ij[::-1]

plt.gray()

fig1 = plt.figure(figsize=(16, 6))

ax1 = plt.subplot(1, 3, 1)
ax2 = plt.subplot(1, 3, 2)
ax3 = plt.subplot(1, 3, 3, sharex=ax2, sharey=ax2)

ax1.imshow(template, cmap=plt.cm.gray)
ax1.set_title('Template')

ax2.imshow(image, cmap=plt.cm.gray)
ax2.set_title('Image')

print(template.shape)

htemp, wtemp = template.shape

rect = plt.Rectangle(
   (x, y),
   wtemp,
```

```
    htemp,
    edgecolor='r',
    facecolor='none'
)

ax2.add_patch(rect)

ax3.imshow(result)
ax3.set_title('Match Template Result')
ax3.autoscale(False)
ax3.plot(
    x, y, 'o',
    markeredgecolor='r',
    markerfacecolor='none',
    markersize=10
)

fig2 = plt.figure(figsize=(10, 20))

ax21 = plt.subplot(1, 2, 1)
ax22 = plt.subplot(1, 2, 2, sharex=ax21, sharey=ax21)

kernel = np.array([
    [0, 1, 0],
    [1, -4, 1],
    [0, 1, 0]
])

edged = np.clip(convolve2d(image, kernel), 0, 1)

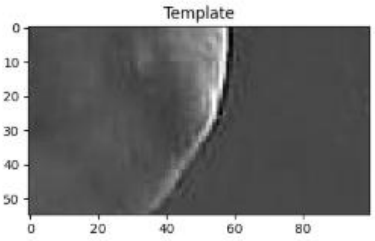
ax21.imshow(image, cmap=plt.cm.gray)
ax21.set_title('Image')

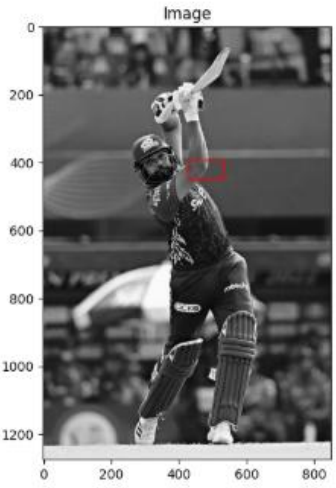
ax22.imshow(edged, cmap=plt.cm.gray)
ax22.set_title('Convolved Image')

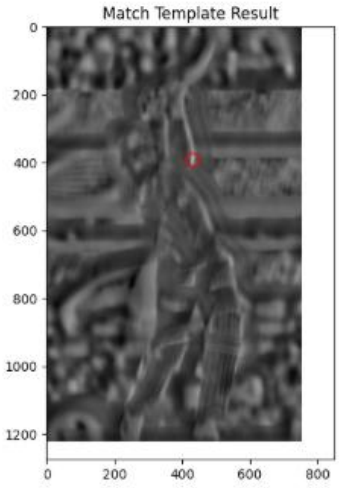
plt.show()
```

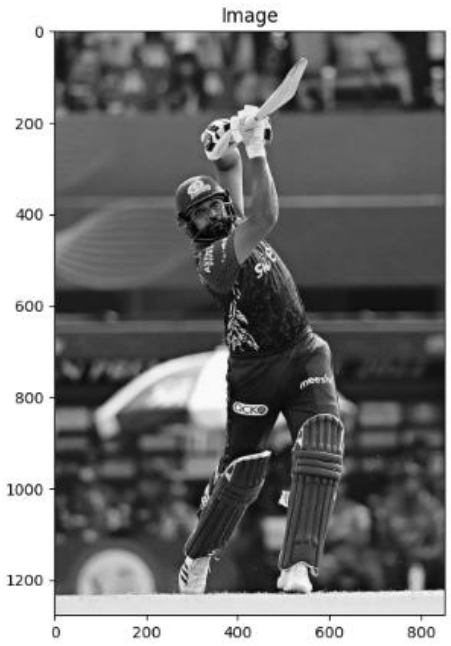
Output :

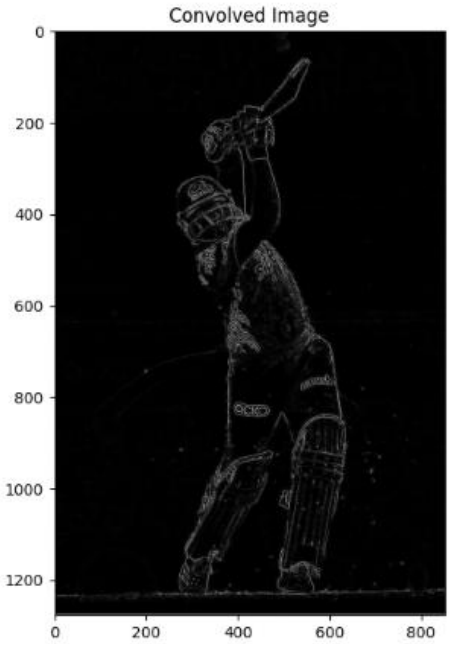
```
(55, 100)
<Figure size 640x480 with 0 Axes>
```











Practical No 4

Aim: - Write program to implement point/pixel intensity transformations such as:
1. Log and Power-law transformations
2. Contrast adjustments
3. Histogram Equalization
4. Thresholding, and halftoning operations

Code:

```
import numpy as np
from skimage import exposure
from skimage.color import rgb2gray
from PIL import Image
import matplotlib.pyplot as plt

img = Image.open(r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg")

plt.figure(figsize=(14, 20))

# Original Image
plt.subplot(3, 3, 1)
plt.imshow(img)
plt.title('Original Image')
plt.gray()

# Log Transform
plt.subplot(3, 3, 2)
im = img.convert('RGB').point(lambda i: int(255 * np.log1p(i / 255) / np.log1p(1)))
plt.imshow(im)
plt.title('Log Transformed')

# Power Law Transform
gamma = 0.5
plt.subplot(3, 3, 3)
im = img.convert('RGB').point(lambda i: int(255 * (i / 255) ** gamma))
plt.imshow(im)
plt.title('Power Transformed')

# Contrast Adjustments
def contrast(c):
    return 0 if c < 70 else (255 if c > 150 else (255 * c - 19950) / 80)

plt.subplot(3, 3, 4)
im = img.convert('RGB').point(contrast)
plt.imshow(im)
plt.title('Contrast Stretching Transformed')

# Histogram Equalization
img_gray = rgb2gray(np.array(img.convert('RGB')))
```

```
img_eq = exposure.equalize_hist(img_gray)
img_aeq = exposure.equalize_adapthist(img_gray, clip_limit=0.03)

plt.subplot(3, 3, 5)
plt.imshow(img_eq, cmap='gray')
plt.title('Histogram Equalized Transformed')

# Adaptive Histogram Equalization
plt.subplot(3, 3, 8)
plt.imshow(img_aeq, cmap='gray')
plt.title('Adaptive Histogram Equalization')

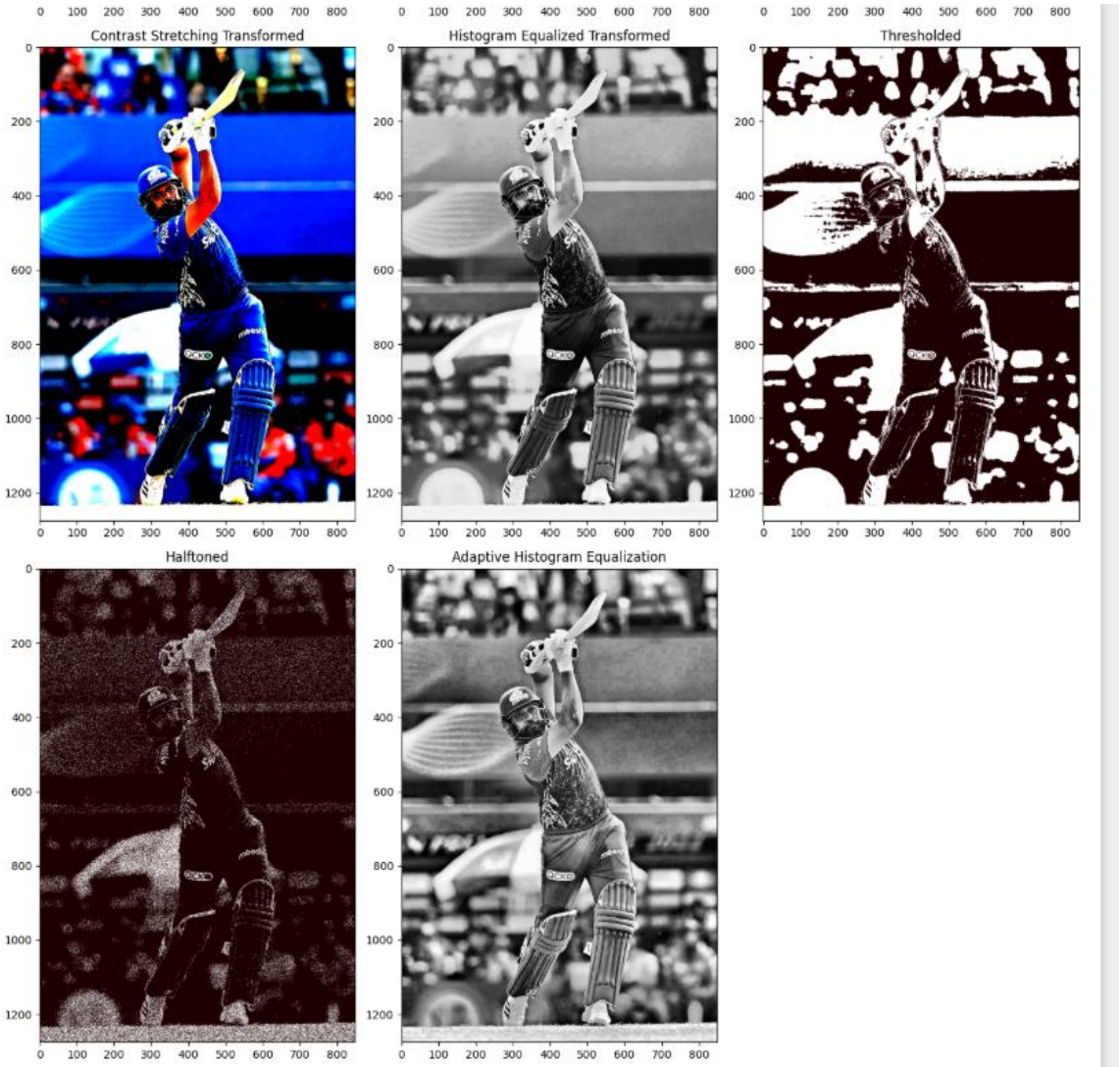
# Thresholding
plt.subplot(3, 3, 6)
im = img.convert('L').point(lambda i: 255 if i > 90 else 0)
plt.imshow(im, cmap='pink')
plt.title('Thresholded')

# Halftoning Operation
plt.subplot(3, 3, 7)
im = np.array(img.convert('L'))
noise = np.random.randint(-128, 128, (im.shape[0], im.shape[1]))
im = Image.fromarray(np.clip(im.astype(int) + noise, 0, 255).astype(np.uint8))
im = im.point(lambda i: 255 if i > 200 else 0)
plt.imshow(im, cmap='pink')
plt.title('Halftoned')

plt.tight_layout()
plt.show()
```

Output :




Contrast Stretching Transformed
Histogram Equalized Transformed
Thresholded
Halftoned
Adaptive Histogram Equalization

Practical No 5

Aim: - Write a program to apply various enhancements on images using image derivatives by implementing Gradient and Laplacian operations.

Code :-

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt
import scipy as sp

def plot_image(image, title):
   plt.imshow(image, cmap='gray')
   plt.title(title, size=20)
   plt.axis('off')

plt.figure(figsize=(15, 10))
plt.gray()

# Read the image and convert it to grayscale
image = ski.color.rgb2gray(
   ski.io.imread(
      r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"
   )
)

plt.subplot(2, 3, 1)
plot_image(image, 'Original Image')

# X-dimension gradient
plt.subplot(2, 3, 2)
ker_x = [[-1, 1]]
derive_x = sp.signal.convolve2d(image, ker_x, mode='same')
plot_image(derive_x, 'Gradient X')

# Y-dimension gradient
plt.subplot(2, 3, 3)
ker_y = [[-1], [1]]
derive_y = sp.signal.convolve2d(image, ker_y, mode='same')
plot_image(derive_y, 'Gradient Y')

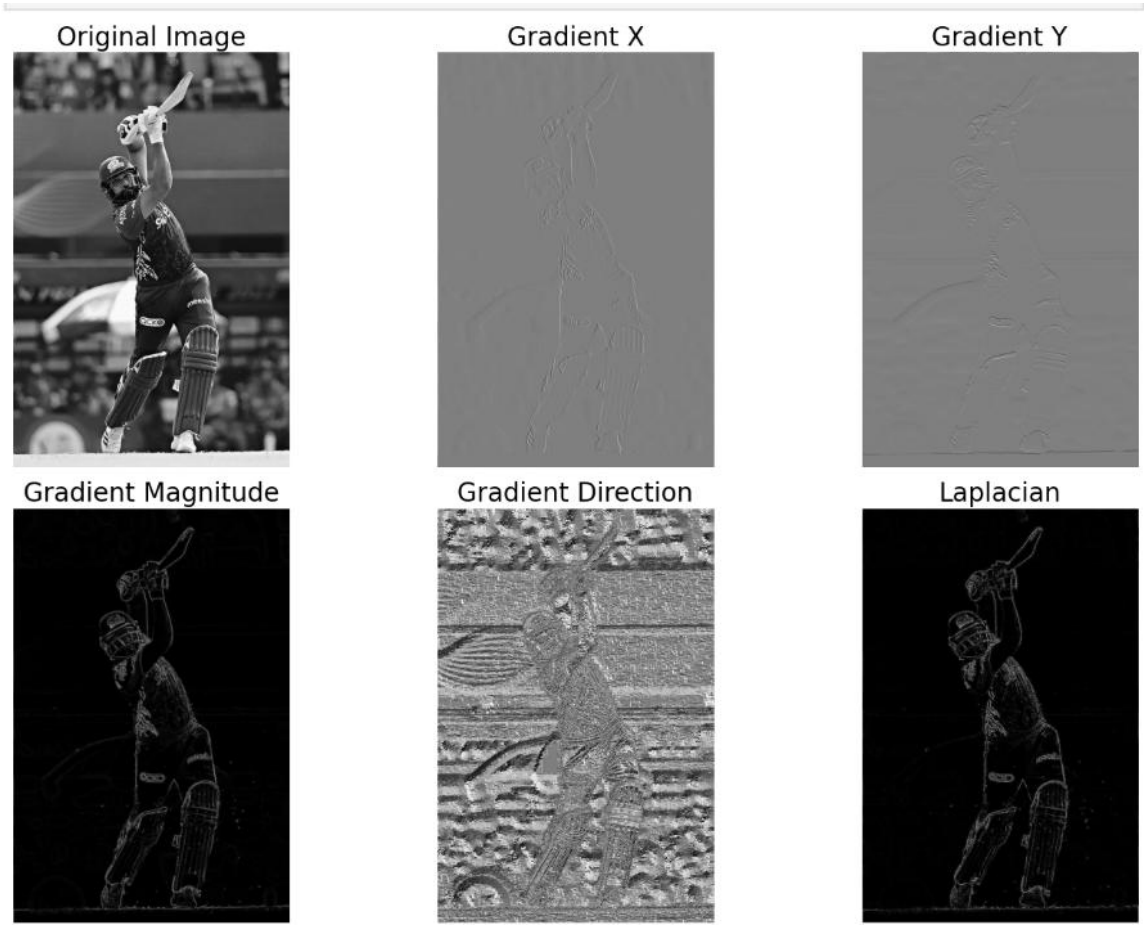
# Gradient magnitude
plt.subplot(2, 3, 4)
im_mag = np.sqrt(derive_x**2 + derive_y**2)
plot_image(im_mag, 'Gradient Magnitude')
```

```
# Gradient direction
plt.subplot(2, 3, 5)
im_dir = np.arctan2(derive_y, derive_x)
plot_image(im_dir, 'Gradient Direction')

# Laplacian
plt.subplot(2, 3, 6)
ker_laplace = [
   [0, -1, 0],
   [-1, 4, -1],
   [0, -1, 0]
]
lapim = np.clip(
   sp.signal.convolve2d(image, ker_laplace, mode='same'),
   0, 1
)
plot_image(lapim, 'Laplacian')

plt.tight_layout()
plt.show()
```

Output :-



Practical No 6

Aim: - Write a program to implement linear and nonlinear noise smoothing on

suitable image or sound signal.

Code :-

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.cvtColor(
   cv.imread(r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"),
   cv.COLOR_BGR2GRAY
)

plt.figure(figsize=(18, 10))

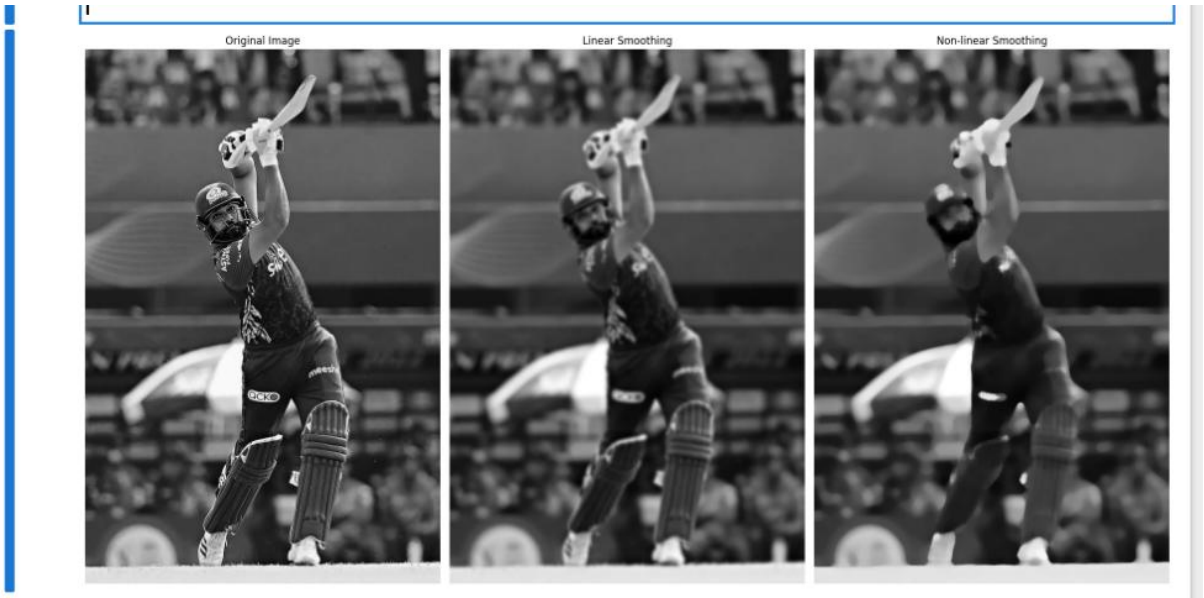
# Original Image
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

# Linear Noise Smoothing
lsimg = cv.GaussianBlur(img, (21, 21), 0)
plt.subplot(1, 3, 2)
plt.imshow(lsimg, cmap='gray')
plt.title('Linear Smoothing')
plt.axis('off')

# Non-linear Noise Smoothing
nlsimg = cv.medianBlur(img, 21)
plt.subplot(1, 3, 3)
plt.imshow(nlsimg, cmap='gray')
plt.title('Non-linear Smoothing')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output :-



Practical No 7

Aim: - Write a program to apply various image enhancement using imagederivatives byimplementing smoothing, sharpening, and unsharp maskingfilters for generating suitable images for specific application requirements.

Code:

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt

def plot_image(image, title):
  plt.imshow(image, cmap='gray')
  plt.title(title)
  plt.axis('off')

# Read image and convert to grayscale
im = ski.color.rgb2gray(
  ski.io.imread(
    r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"
  )
)

plt.figure(figsize=(15, 9))
plt.gray()

# Original Image
plt.subplot(1, 4, 1)
plot_image(im, 'Original Image')

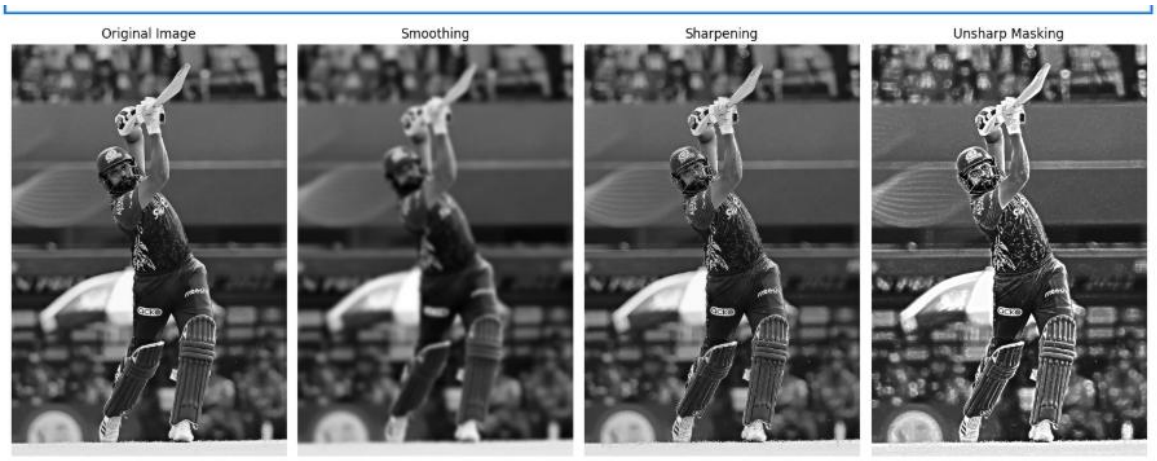
# Smoothing
plt.subplot(1, 4, 2)
gausmoothim = ski.filters.gaussian(im, sigma=5)
plot_image(gausmoothim, 'Smoothing')

# Sharpening
plt.subplot(1, 4, 3)
lapsharpim = np.clip(im + ski.filters.laplace(im), 0, 1)
plot_image(lapsharpim, 'Sharpening')

# Unsharp Masking
plt.subplot(1, 4, 4)
usmaskim = np.clip(
  im + np.clip(im - gausmoothim, 0, 1) * 5,
  0, 1
)
plot_image(usmaskim, 'Unsharp Masking')
```

```
plt.tight_layout()
plt.show()
```

Output:



Practical No 8

Aim: - Write a program to Apply edge detection techniques such as Sobel and Canny to extract meaningful information from the given image samples.

Code:

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt

def plot_image(image, title):
  plt.imshow(image, cmap='gray')
  plt.title(title)
  plt.axis('off')

# Load your Rohit Sharma image
im = ski.color.rgb2gray(
  ski.io.imread(
    r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"
  )
)

plt.figure(figsize=(11, 9))

# Original Image
plt.subplot(2, 3, 1)
plot_image(im, 'Original Image')

# Sobel
plt.subplot(2, 3, 2)
plot_image(ski.filters.sobel(im), 'Sobel Operated')

# Roberts
plt.subplot(2, 3, 3)
plot_image(ski.filters.roberts(im), 'Roberts Operated')

# Scharr
plt.subplot(2, 3, 4)
plot_image(ski.filters.scharr(im), 'Scharr Operated')

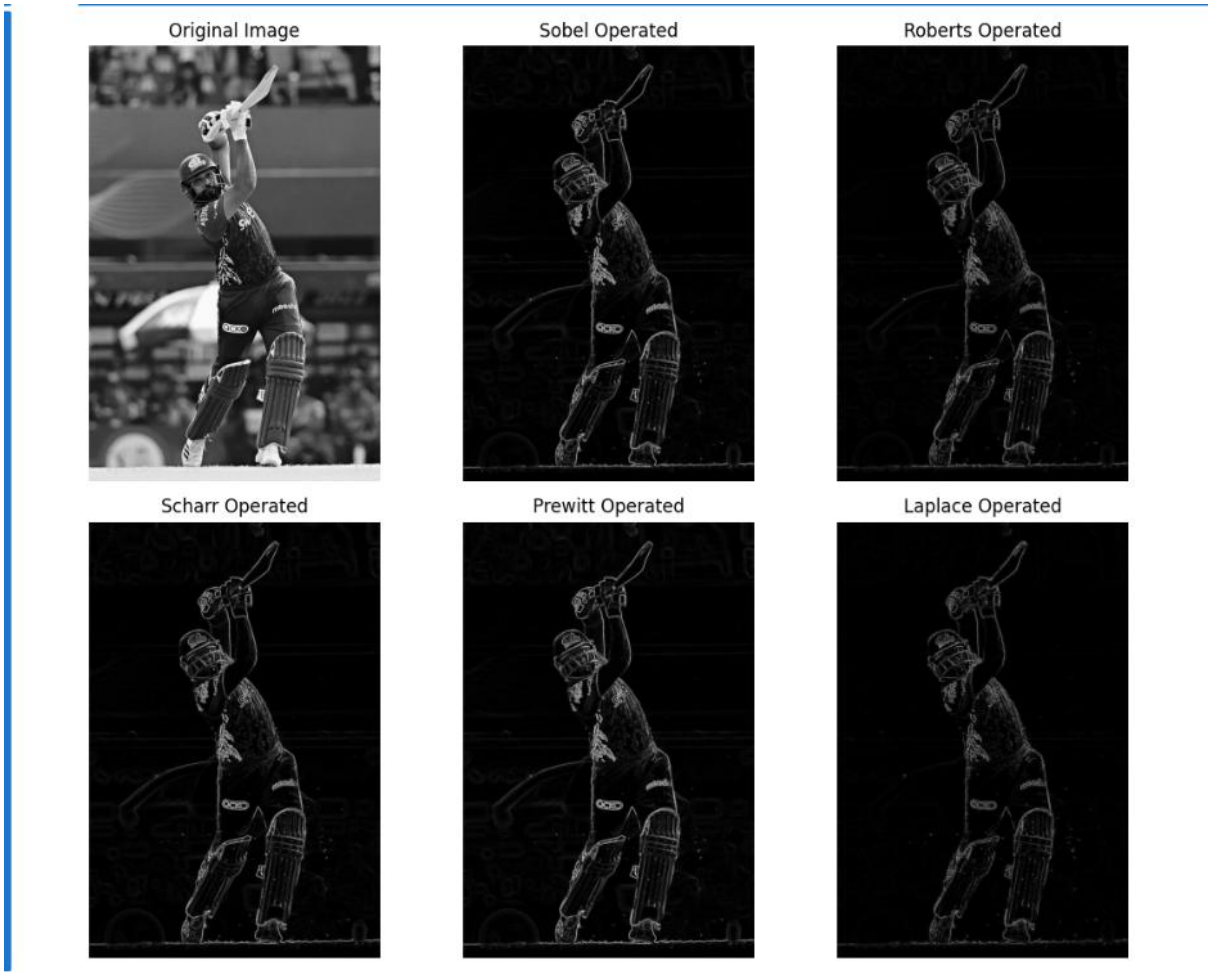
# Prewitt
plt.subplot(2, 3, 5)
plot_image(ski.filters.prewitt(im), 'Prewitt Operated')

# Laplace
plt.subplot(2, 3, 6)
plot_image(
```

```
    np.clip(ski.filters.laplace(im), 0, 1),
    'Laplace Operated'
)

plt.tight_layout()
plt.show()
```

Output :



Practical No 9

Aim: - Write a program to implement various morphological image processing techniques.

Code:

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt

# Function to display images
def plot_image(image, title=""):
    plt.imshow(image, cmap='gray')
    plt.title(title)
    plt.axis('off')

# Read your Rohit Sharma image and convert to grayscale
img = ski.io.imread(
    r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"
)
img = ski.color.rgb2gray(img)

# Convert grayscale image to binary
imbin = img > 0.4

plt.figure(figsize=(12, 16))

# Original Binary Image
plt.subplot(4, 3, 1)
plot_image(imbin, 'Original Image')

# Erosion
plt.subplot(4, 3, 2)
eroded = ski.morphology.binary_erosion(
    imbin, ski.morphology.disk(5)
)
plot_image(eroded, 'Erosion')

# Dilation
plt.subplot(4, 3, 3)
dilated = ski.morphology.binary_dilation(
    imbin, ski.morphology.disk(5)
)
plot_image(dilated, 'Dilation')

# Convex Hull
plt.subplot(4, 3, 4)
chull = ski.morphology.convex_hull_image(imbin)
plot_image(chull, 'Convex Hull')
```

```
# Closing
plt.subplot(4, 3, 5)
closed = ski.morphology.binary_closing(
    imbin, ski.morphology.disk(5)
)
plot_image(closed, 'Closing')

# Opening
plt.subplot(4, 3, 6)
opened = ski.morphology.binary_opening(
    imbin, ski.morphology.disk(5)
)
plot_image(opened, 'Opening')

# Skeletonizing
plt.subplot(4, 3, 7)
skeleton = ski.morphology.skeletonize(imbin)
plot_image(skeleton, 'Skeletonized')

# White Top-Hat
plt.subplot(4, 3, 8)
wth = ski.morphology.white_tophat(
    imbin, ski.morphology.disk(5)
)
plot_image(wth, 'White Top-Hat')

# Black Top-Hat
plt.subplot(4, 3, 9)
bth = ski.morphology.black_tophat(
    imbin, ski.morphology.disk(5)
)
plot_image(bth, 'Black Top-Hat')

# Remove Small Objects
plt.subplot(4, 3, 10)
rso = ski.morphology.remove_small_objects(
    imbin, min_size=500
)
plot_image(rso, 'Small Objects Removed')

# Extract Boundary
plt.subplot(4, 3, 11)
be = imbin & ~ski.morphology.binary_erosion(imbin)
plot_image(be, 'Boundary Extracted')

# Beucher Gradient
plt.subplot(4, 3, 12)
footprint = ski.morphology.footprint_rectangle((3, 3))
mg = (
    ski.morphology.dilation(img, footprint)
```

```
    - ski.morphology.erosion(img, footprint)
)
plot_image(mg, 'Beucher Gradient')

plt.tight_layout()
plt.show()
```

Output :



Practical No 10

Aim: - Write a program to extract image features by implementing methods like

corner and blob detectors, HoG and Haar features.

Code :

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt

# Function to display images
def plot_image(image, title=""):
    plt.title(title)
    plt.imshow(image)
    plt.axis('off')

# Read your Rohit Sharma image
img = ski.io.imread(
    r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"
)

# Convert image to grayscale
imgray = ski.color.rgb2gray(img)

plt.figure(figsize=(14, 12))
plt.gray()

# 1. Original Image
plt.subplot(2, 3, 1)
plot_image(img, 'Original Image')

# 2. Harris Corner Detector
plt.subplot(2, 3, 2)
coordinates = ski.feature.corner_harris(imgray, k=0.001)
hcdim = img.copy()
hcdim[coordinates > 0.01 * coordinates.max()] = [255, 0, 0]
plot_image(hcdim, 'Harris Corner')

# 3. Histogram of Oriented Gradients (HoG)
plt.subplot(2, 3, 3)
fd, hog_image = ski.feature.hog(
    imgray,
    orientations=8,
    pixels_per_cell=(16, 16),
    cells_per_block=(1, 1),
    visualize=True
)
hog_image = ski.exposure.rescale_intensity(
    hog_image, in_range=(0, 20)
```

```
)
plot_image(hog_image, 'HoG')

# 4. Laplacian of Gaussian (LoG) Blob Detector
log_blobs = ski.feature.blob_log(
    imgray, max_sigma=30, num_sigma=10, threshold=0.1
)
log_blobs[:, 2] = np.sqrt(2) * log_blobs[:, 2]

plt.subplot(2, 3, 4)
plot_image(img, 'LoG')
ax = plt.gca()
for y, x, rad in log_blobs:
    circle = plt.Circle(
        (x, y), rad, color='red', linewidth=2, fill=False
    )
    ax.add_patch(circle)

# 5. Difference of Gaussian (DoG) Blob Detector
dog_blobs = ski.feature.blob_dog(
    imgray, max_sigma=30, threshold=0.1
)
dog_blobs[:, 2] = np.sqrt(2) * dog_blobs[:, 2]

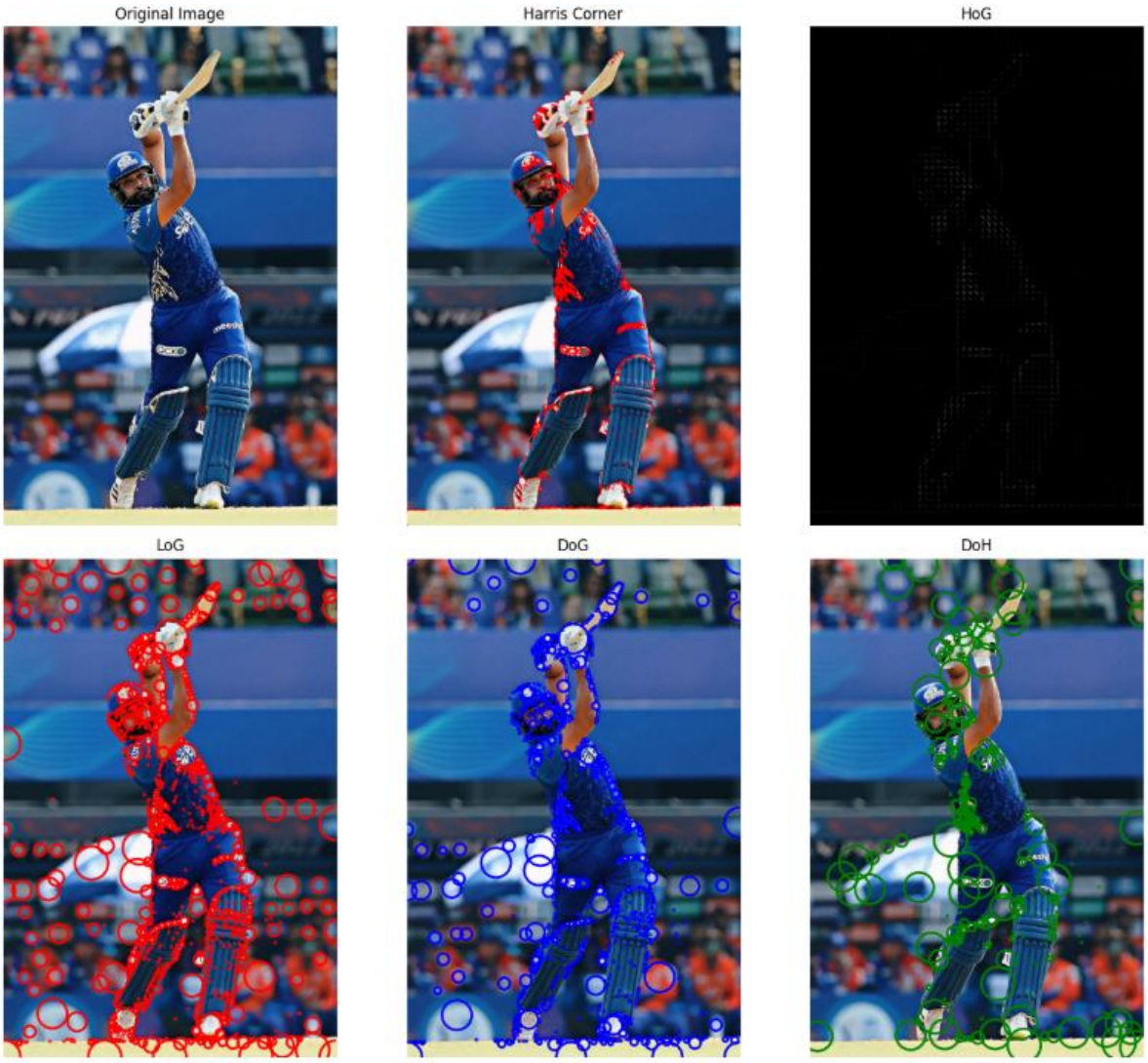
plt.subplot(2, 3, 5)
plot_image(img, 'DoG')
ax = plt.gca()
for y, x, rad in dog_blobs:
    circle = plt.Circle(
        (x, y), rad, color='blue', linewidth=2, fill=False
    )
    ax.add_patch(circle)

# 6. Determinant of Hessian (DoH) Blob Detector
doh_blobs = ski.feature.blob_doh(
    imgray, max_sigma=30, num_sigma=10, threshold=0.005
)
doh_blobs[:, 2] = np.sqrt(2) * doh_blobs[:, 2]

plt.subplot(2, 3, 6)
plot_image(img, 'DoH')
ax = plt.gca()
for y, x, rad in doh_blobs:
    circle = plt.Circle(
        (x, y), rad, color='green', linewidth=2, fill=False
    )
    ax.add_patch(circle)

plt.tight_layout()
plt.show()
```

Output :



Practical No 11

Aim: - Write a program to apply segmentation for detecting lines, circles, and other shapes/ objects. Also, implement edge-based and region-based segmentation.

Code :-

```
import numpy as np
import skimage as ski
import matplotlib.pyplot as plt
import scipy as sp

# Function to display images
def plot_image(image, title=""):
    plt.imshow(image, cmap='gray')
    plt.title(title)
    plt.axis('off')

# Read image and convert to grayscale
image = ski.io.imread(
    r"C:\Users\rutik\Downloads\59277938-cricketer-rohit-sharma-photo.jpg"
)
image = ski.color.rgb2gray(image)

plt.figure(figsize=(15, 12))

# 1. Original Image
plt.subplot(2, 3, 1)
plot_image(image, 'Original Image')

# 2. Straight-Line Hough Transform
plt.subplot(2, 3, 2)
edges = ski.feature.canny(image, sigma=2.5)
h, theta, d = ski.transform.hough_line(edges)

for _, angle, dist in zip(
    *ski.transform.hough_line_peaks(h, theta, d)
):
    y0 = (dist - 0 * np.cos(angle)) / np.sin(angle)
    y1 = (
        dist - image.shape[1] * np.cos(angle)
    ) / np.sin(angle)

    plt.plot((0, image.shape[1]), (y0, y1), '-r')

plot_image(image, 'Line Segmentation')
plt.subplot(2, 3, 2).set_xlim((0, image.shape[1]))
plt.subplot(2, 3, 2).set_ylim((image.shape[0], 0))
```

```
# 3. Circle Hough Transform
plt.subplot(2, 3, 3)

# Use edge image for circle detection
hough_radii = np.arange(20, 76, 2)
hough_res = ski.transform.hough_circle(edges, hough_radii)

accums, c_x, c_y, radii = ski.transform.hough_circle_peaks(
    hough_res, hough_radii, total_num_peaks=8
)

img_rgb = ski.color.gray2rgb(image)

for center_y, center_x, radius in zip(c_y, c_x, radii):
    circle_y, circle_x = ski.draw.circle_perimeter(
        center_y, center_x, radius, shape=img_rgb.shape[:2]
    )
    img_rgb[circle_y, circle_x] = (1, 0, 0)

plot_image(img_rgb, 'Circle Segmentation')

# 4. Edge-Based Segmentation
plt.subplot(2, 3, 4)
edges = ski.feature.canny(image, sigma=2)

filled_edges = sp.ndimage.binary_fill_holes(edges)
imces = ski.morphology.remove_small_objects(
    filled_edges.astype(bool), min_size=50
)

plot_image(imces, 'Edge-Based Segmentation')

# 5. Region-Based Segmentation
plt.subplot(2, 3, 5)
gradient = ski.filters.sobel(image)

segments_watershed = ski.segmentation.watershed(
    gradient, markers=500, compactness=0.00001
)

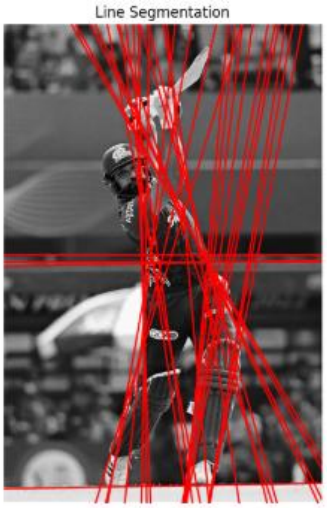
marked_image = ski.segmentation.mark_boundaries(
    image, segments_watershed, color=(1, 0, 0)
)

plot_image(marked_image, 'Region-Based Segmentation')

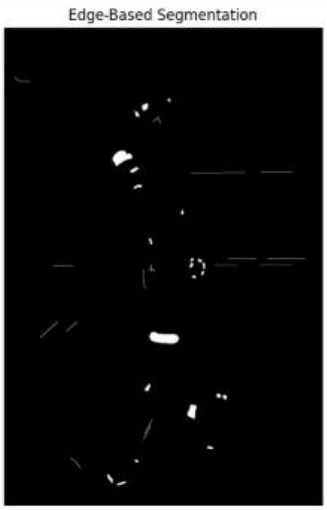
plt.tight_layout()
plt.show()
```

Output :


Original Image


Line Segmentation


Circle Segmentation


Edge-Based Segmentation


Region-Based Segmentation